CENTRO DE ENSEÑANZA TÉCNICA Y SUPERIOR



LENGUAJES DE PROGRAMACIÓN ORIENTADA A OBJETOS
Profra. Vanessa Miranda

ProyectoFinal_ICC202_2026
Avance 2: Implementación básica y funcionalidad

Presentado por:

Omar Muñoz - 36471
Paul Alvarez - 40683
Karim Herrera - 39219
Abel Martinez - 38531

Tijuana, B.C. a 21 de abril de 2026

1. Resumen

En este segundo avance presentamos la implementación del 25% de la interfaz SFML, el sistema completo de clases base con herencia y polimorfismo, el motor de combate por turnos funcional, y el sistema de reliquias con rareza. El proyecto ha avanzado significativamente desde la propuesta inicial, logrando un prototipo simulable que demuestra las mecánicas centrales del juego.

2. Objetivos Cumplidos

2.1 Jerarquía de Base de Clases

Se implementó la jerarquía completa de clases aplicando herencia y polimorfismo:

• Entity (clase base abstracta): Contiene las 6 estadísticas de combate (HP, ATK, DEF, SP.ATK, SP.DEF, SPD), sistema de 4 movimientos, manejo de bloqueo, y métodos virtuales para displayStats().

• Player: Hereda de Entity e implementa sistema de niveles, experiencia, inventario de reliquias y pociones con límite de slots expandible.

• Enemy: Hereda de Entity e incluye tipos de enemigo (Sentinel, Shade, Wraith, Golem, Shaman, Specter), sistema de move pools para selección aleatoria de movimientos, y cálculo dinámico de recompensas de EXP.

• MiniBoss: Hereda de Enemy con movimientos hardcodeados más poderosos y multiplicador de EXP de 1.5x.

2.2 Sistema de Movimientos

La clase Move implementa un sistema flexible de ataques y efectos:

```
enum class MoveType { PHYSICAL, SPECIAL, EFFECT };
enum class MoveEffect { NONE, HEAL, BLOCK, ATK_UP, ATK_DOWN, ... };
enum class MoveTarget { SELF, ENEMY };
```

Cada movimiento tiene poder, precisión, efecto, potencia del efecto (en porcentaje), y objetivo. Esto permite crear movimientos como 'War Cry' que aumenta ATK en 40% o 'Crush' que hace 80 de daño base con 90% de precisión.

2.3 Reliquias

Se implementaron 4 tipos de reliquias, todas heredando de la clase base abstracta Relic:

- StatRelic: Aumenta permanentemente una estadística del jugador (HP, ATK, DEF, etc.)
- MoveRelic: Reemplaza un movimiento existente o aumenta su poder si ya lo posee
- ProbabilityRelic: Modifica probabilidades de encuentros corruptos, drops de mini-bosses, o precisión de movimientos (WIP)

- UtilityRelic: Funcionalidades misceláneas como expandir slots de pociones, curación al entrar a pisos, o boost de EXP

Cada reliquia tiene rareza (Común, Rara, Legendaria) que determinará su distribución en un sistema de drops.

2.4 Motor de Combate

CombatEngine maneja la lógica completa de combate:

```
int CombatEngine::calculateDamage(Entity* attacker, Entity* defender, Move move) {
    int attackStat = (move.getType() == PHYSICAL) ? atk : spAtk;
    int defenseStat = (move.getType() == PHYSICAL) ? def : spDef;
    if (defender->getIsBlocking()) defenseStat *= 1.5;
    return ((attackStat * power) / defenseStat) / 2;
}
```

El sistema implementa orden de turnos basado en SPD, checks de precisión, aplicación de efectos (buffs/debuffs), manejo de bloqueo, y reset automático de estadísticas al finalizar combate.

2.5 Interfaz SFML Preliminar (25%)

Se implementó una interfaz básica que muestra:

• Ventana gráfica de 800x600 píxeles
• Título del juego centrado
• Estadísticas del jugador (lado izquierdo en verde) mostrando nombre, HP actual/máximo, ATK, DEF y SPD
• Estadísticas del enemigo (lado derecho en rojo) con el mismo formato
• Log de combate en amarillo mostrando el desarrollo de cada turno
• Texto de resultado indicando el ganador al finalizar
• Control mediante tecla ESPACIO para avanzar turnos (movimientos aleatorios por ahora)

La interfaz es funcional pero básica, se planea agregar botones interactivos para selección de movimientos y sprites en avances posteriores.

3. Desafíos Enfrentados y Soluciones

3.1 Configuración del Entorno de Desarrollo

Desafío: El equipo trabaja en sistemas operativos diferentes (Windows y macOS). La configuración de SFML, compiladores y VS Code presentó problemas de compatibilidad.

Se trabajó en crear archivos de configuración .json separados para ambos sistemas (tasks.json, launch.json, c_cpp_properties.json). Se documentó el proceso de instalación de MinGW en Windows y Homebrew en macOS. Karim y Abel enfrentaron dificultades para ejecutar el código en sus máquinas, por lo que la mayoría del desarrollo se centralizó en la máquina de Omar donde los .json funcionaban correctamente. Esto implicó coordinación cercana y uso intensivo de Git para que todo el equipo pudiera contribuir.

3.2 Diseño del Sistema de Movimientos

Desafío: Inicialmente, los movimientos de efecto (buffs/debuffs) eran tratados igual sin importar si venían de un enemigo regular o un mini-boss, lo que hacía los mini-bosses menos desafiantes.

Solución: Se agregó el parámetro effectPotency a la clase Move, permitiendo que los mini-bosses tengan efectos más fuertes comparados con enemigos normales. Esto requirió refactorizar todos los constructores de Move en Enemy.cpp y MiniBoss.cpp.

3.3 Manejo de Estadísticas en Combate

Desafío: Las estadísticas modificadas durante el combate (por buffs/debuffs) permanecían alteradas después de la batalla, afectando encuentros futuros.

Solución: Se implementó un sistema de estadísticas base en Entity (baseAtk, baseDef, etc.) y un método resetStats() que restaura todos los valores a su estado original. Esto se ejecuta automáticamente al finalizar cada combate.

3.4 Implementación de Herencia y Polimorfismo

Desafío: El sistema de reliquias requería comportamiento polimórfico donde cada tipo de reliquia aplica efectos diferentes al jugador, pero todas debían almacenarse en un solo vector.

Solución: Se diseñó Relic como clase base abstracta con el método virtual applyEffect(Player&). Cada subclase (StatRelic, MoveRelic, etc.) implementa su propia versión. El jugador almacena reliquias como vector<Relic*>, permitiendo polimorfismo en runtime. Se utilizó dynamic_cast para verificar tipos específicos cuando es necesario acceder a métodos únicos de subclases, aunque al final no se acabó implementando por ahora pero probablemente sea necesario en el futuro.

4. Distribución del Trabajo

Omar Muñoz: Diseño e implementación de la jerarquía de clases (Entity, Player, Enemy, MiniBoss), sistema de movimientos completo, CombatEngine, colaboracion en las clases de reliquias, coordinación del repositorio Git y resolución de merge conflicts. Fue la máquina principal de desarrollo debido a compatibilidad de .json.

Paul Alvarez: Implementación de la interfaz SFML preliminar en main.cpp, sistema de funcionalidades de movimientos para Player (initializeMovePool, selectMovesFromPool), configuración de archivos .json para compatibilidad Mac/Windows, debugging de problemas de compilación multiplataforma.

Karim Herrera y Abel Martinez: Implementación del sistema de pociones (Potion.h, Potion.cpp), colaboración en clases de reliquias (StatRelic, MoveRelic, ProbabilityRelic, UtilityRelic), creación de varios movimientos para diferentes tipos de enemigos, correcciones de bugs menores. Enfrentaron dificultades técnicas con la configuración local que limitaron su capacidad de push directo, por lo que coordinaron con Omar para integrar su código.

5. Diagrama UML

Se creó un diagrama UML en Miro que ilustra:

- Jerarquía de herencia: Entity -> Player, Entity -> Enemy -> MiniBoss
- Jerarquía de reliquias: Relic -> StatRelic, MoveRelic, ProbabilityRelic, UtilityRelic
- Composición: Player contiene Move[], Relic[], Potion[]
- Dependencias: CombatEngine usa Player y Entity
- Atributos y métodos principales de cada clase con especificación de visibilidad

El diagrama está disponible en el repositorio del proyecto y puede requerir ajustes menores para mayor claridad visual.

6. Estado Actual y Próximos Pasos

6.1 Funcionalidades Completadas

Jerarquía completa de clases con herencia y polimorfismo
Sistema de combate por turnos 100% funcional
Sistema de movimientos con tipos físicos, especiales y de efecto
Clases de reliquias con 4 categorías y rareza
Clases de pociones con efectos temporales
6 tipos de enemigos con move pools únicos
Mini-bosses con movimientos hardcodeados y EXP mejorado

Interfaz SFML básica con visualización de combate
Repositorio Git configurado y funcional

6.2 Pendientes para Siguiente Avance

- Sistema de pisos y puertas
- Puertas corruptas con probabilidad de mini-boss
- Sistema de recompensas post-combate
- Interfaz mejorada al 50% (botones interactivos, sprites)
- Sistema de sinergias entre reliquias
- Boss final con sistema de fases
- Añadir Block con prioridad en combate
- Llamado de pociones
- Ataques criticos

7. Conclusión

El segundo avance representa un progreso significativo en el desarrollo de The Catacombs. Hemos construido una base sólida de programación orientada a objetos con herencia y polimorfismo bien implementados, un motor de combate completamente funcional, y un sistema de reliquias robusto. Aunque enfrentamos desafíos técnicos relacionados con la compatibilidad multiplataforma y coordinación del equipo, logramos superarlos mediante comunicación efectiva y adaptación de nuestro flujo de trabajo.

El prototipo actual demuestra que las mecánicas centrales del juego son viables y funcionan como se esperaba. La interfaz SFML, aunque básica, proporciona una visualización clara del combate y sienta las bases para mejoras visuales futuras. El equipo está preparado para continuar con la implementación del sistema de pisos, mejoras a la interfaz, y el boss final en los siguientes avances.

8. Referencias del Código

El código fuente completo está disponible en el repositorio GitHub:
https://github.com/abelmartinez-bit/The-Catacombs

Estructura del proyecto:
• /src - Implementaciones (.cpp)
• /include - Declaraciones de clases (.h)
• /build - Ejecutables compilados
• /.vscode - Configuración de VS Code (Mac, Windows)
• /assets - Recursos

9. Minutas

9.1 Minuta 1



Fecha: 17 de abril de 2026
Modalidad: Presencial
Participantes: Omar Muñoz, Paul Alvarez, Karim Herrera, Abel Martinez

Temas y Acuerdos

- Sistema de entidades (Entity.h - clase base)
- Sistema de reliquias con rareza (Relic.h + tipos específicos)
- Jugador sin arquetipos (Player.h)
- Pociones de combate (Potion.h)
- MoveRelic (modifica movimientos)
- StatRelic (modifica estadísticas)
- ProbabilityRelic (efectos probabilísticos)
- UtilityRelic (utilidades)

9.2 Minuta 2

Fecha: 21 de abril de 2026
Modalidad: Presencial
Participantes: Omar Muñoz, Paul Alvarez, Karim Herrera, Abel Martinez

Temas y Acuerdos

- Mini-bosses (Miniboss.h)
- Polish y refinamiento de UI/UX
- Interfaz gráfica base con SFML
- Jugador con arquetipos y moves (Player.h - Archer, Warrior, etc)
- Enemigos básicos (Enemy.h - Sentinel, etc)
- Motor de combate por turnos (CombatEngine.h)
- Sistema de movimientos/ataques (Move.h)

10. Cronograma General de Actividades

Actividad	Abr S1	Abr S2	Abr S3	May S1	May S2	Responsable
Formación del equipo y propuesta inicial (Avance 1)	▮					Equipo
Repositorio, entorno y diseño UML	▮	▮				Abel / Omar / Paul
Clases base, motor de combate e interfaz SFML (25%)		▮	▮			Omar / Paul
Reliquias, rareza y mini-bosses		▮	▮			Omar / Karim / Abel
Avance 2 — 22 de abril			▮			Equipo
Pisos, puertas, sinergias e interfaz 50%			▮	▮		Karim / Abel
Pruebas por módulo y correcciones				▮	▮	Paul
Avance 3 — 8 de mayo				▮		Equipo
Boss final con fases y librería propia				▮	▮	Karim / Omar
Interfaz 90% e integración final					▮	Abel
Avance 4 — 22 de mayo					▮	Equipo